

CSc 8830 Computer Vision, Module 4

Finding the Boundary of a Human in RGB and Thermal Images with Classical Computer Vision

Sirichandana Bikkasani
Georgia State University

September 28, 2026

Abstract

This module outlines a person in a normal colour (RGB) image and in a thermal (infrared) image using classical OpenCV only. The segmentation uses thresholding, morphology, connected components, contours and GrabCut, with no deep learning or machine learning. Each result is then placed next to the Segment Anything model, the reference the assignment asks us to compare against. Section 5 works through, with derivations, how edge detection and region segmentation can be done in the Fourier (frequency) domain. A Streamlit web application makes every part reachable and lets the reader run the methods on their own images.

Live web app: <https://csc8830-module4-human-seg.streamlit.app>

Code repository: <https://github.com/siri423/CSc8830-Computer-Vision>

1 Where each requirement is covered

Requirement	Where
Q1, human boundary in an RGB image (classical)	<code>src/rgb.segment.py</code> , app tab RGB
Q2, human boundary in a thermal image (classical)	<code>src/thermal.segment.py</code> , app tab Thermal
Comparison with SAM	<code>src/compare.py</code> , Section 4, app tab Compare
Q3, Fourier theory for edges and regions	Section 5, <code>src/fourier.py</code> , app tab Theory
Web application demo	<code>app.py</code> (Streamlit)

2 Q1: Human boundary in an RGB image

In a colour photo the person and the background mainly differ in colour, so we use GrabCut [1]. GrabCut fits Gaussian mixtures to the foreground and background colours and cuts the lowest cost boundary between them. It learns those colour statistics from this one image, so it is not a trained model.

The steps are as follows. First we build an automatic seed, called a trimap: a thin outer border is marked as sure background, a central column as probable foreground, and a core torso block as sure foreground, so no clicking is needed. The sure foreground core is what keeps parts whose colour matches the background, such as hair against a beige backdrop, from being removed. Then we run GrabCut. Next we clean the mask with a morphological opening to drop speckle and a

closing to fill small gaps, keep the single largest connected component since a person is one object, and fill interior holes. Finally we trace the outer contour, which is the boundary we report.

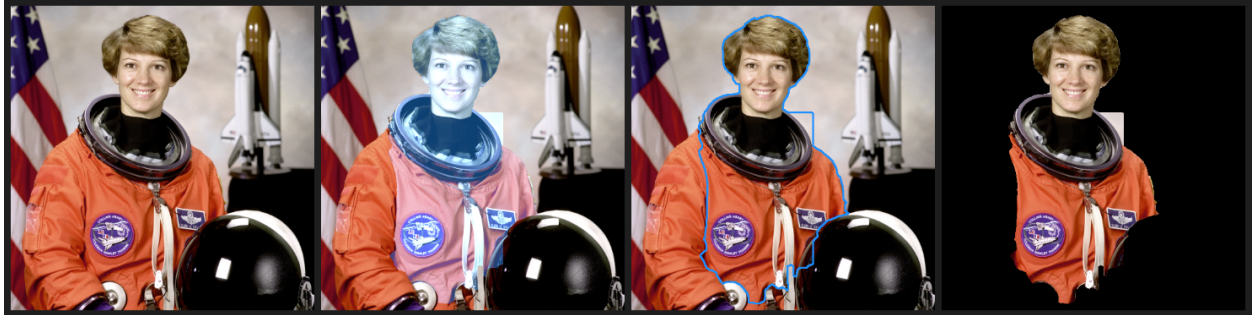


Figure 1: RGB result. From left to right: input, shaded mask, boundary, and the image with the background removed. GrabCut captures the head, face and suit and leaves out the flag, the shuttle model and the black helmet.

3 Q2: Human boundary in a thermal image

In a thermal image a warm body radiates more than the cool ground, so the person is a bright, compact, roughly vertical blob. Colour models are not needed; intensity and shape are enough. The one difficulty is that other things are hot too, such as sunlit walls and lit windows, so the brightest pixel is not always the person. We handle this with a two stage, coarse to fine method.

Stage one locates the person. We blur to reduce sensor noise, threshold at the larger of the Otsu value and a high intensity percentile so only the hottest regions remain, and clean up with morphology. Then we score every hot blob on how human it looks, using shape and context only: an aspect ratio near two to one (an upright person rather than a thin window), a ground plane prior (people stand low in the frame, not high on a building), isolation (a person is ringed by cool ground, whereas a window sits inside a warm wall), and size and solidity. The best scoring blob is the person.

Stage two refines the outline. We crop a box around the located person, extended downward so the dimmer legs are included, and run a local Otsu threshold inside it. Because that box leaves out the far away hot windows, a lower local threshold safely recovers the whole body. We then fill holes and trace the contour.



Figure 2: Thermal result. From left to right: input, shaded mask, boundary, and the background removed image. The pedestrian is outlined from head to feet while the warm building windows are ignored.

4 Comparison with SAM

We compare each classical mask against the Segment Anything demo run with the text prompt **person**. SAM is used only as the reference; our own code never uses it. We report three numbers: the Intersection over Union (IoU, Jaccard), the Dice coefficient (F1), and a boundary F-score that matches the two outlines within a two pixel tolerance. The boundary F-score is the one that speaks directly to exact boundary quality.

Image	IoU	Dice	Boundary-F @2px
RGB (astronaut)	0.68	0.81	0.52
Thermal (pedestrian)	0.82	0.90	0.95

Table 1: Agreement of our classical masks with SAM.

On the thermal image the classical pipeline almost matches SAM, with an IoU of 0.82 and a boundary F-score of 0.95. A warm body against a cool background is a high contrast case that thresholding handles well. On the RGB image the classical result is weaker, with an IoU of 0.68 and a boundary F-score of 0.52. GrabCut recovers the person as a whole, but where foreground and background colours are similar, such as hair against the beige backdrop or the arm against the suit, its boundary drifts, while SAM tracks the true edge. This is the trade off the exercise is about: classical methods are simple and need no training, while a learned model like SAM follows hard boundaries that colour or intensity cues alone cannot resolve.

5 Q3: Edge detection and region segmentation in the Fourier domain

The transform. For an image $f(x, y)$ the two dimensional Fourier transform is

$$F(u, v) = \iint_{-\infty}^{\infty} f(x, y) e^{-j2\pi(ux+vy)} dx dy, \quad (1)$$

with inverse $f(x, y) = \iint F(u, v) e^{j2\pi(ux+vy)} du dv$. Slowly varying, smooth regions concentrate their energy at low frequencies (small $D = \sqrt{u^2 + v^2}$), while sharp changes, which is what edges and fine texture are, contribute high frequencies (large D).

5.1 Edge detection is high-pass filtering

Edges are places where f changes fast, meaning large derivatives. The derivative theorem of the Fourier transform says

$$\mathcal{F}\left\{\frac{\partial f}{\partial x}\right\} = j2\pi u F(u, v), \quad \mathcal{F}\left\{\frac{\partial f}{\partial y}\right\} = j2\pi v F(u, v). \quad (2)$$

Taking a derivative multiplies the spectrum by a factor proportional to frequency, so it lifts high frequencies and holds down low ones. Every gradient based edge operator is therefore a high-pass filter. For the Laplacian $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$,

$$\mathcal{F}\{\nabla^2 f\} = (j2\pi u)^2 F + (j2\pi v)^2 F = -4\pi^2 (u^2 + v^2) F(u, v) = -4\pi^2 D^2 F(u, v), \quad (3)$$

so the Laplacian is the high-pass transfer function $H(u, v) = -4\pi^2 D^2$. To find edges we keep the high frequencies, for example with a Gaussian high-pass filter

$$H_{\text{hp}}(u, v) = 1 - e^{-D^2/(2D_0^2)}, \quad (4)$$

form $G(u, v) = F(u, v) H_{\text{hp}}(u, v)$, and inverse transform. This is valid because of the convolution theorem: convolving the image with an edge kernel $h(x, y)$ in space is the same as multiplying spectra in frequency,

$$g = f * h \iff G(u, v) = F(u, v) H(u, v). \quad (5)$$

So applying an edge filter and multiplying by $H(u, v)$ are the same operation.

5.2 Laplacian of Gaussian, a band-pass edge detector

Differentiation also lifts noise, which lives at high D , so we smooth first with a Gaussian g_σ , whose transform is again a Gaussian, $\mathcal{F}\{g_\sigma\} = e^{-2\pi^2\sigma^2 D^2}$. Combining this with the Laplacian in (3) gives the Laplacian of Gaussian (LoG) transfer function

$$H_{\text{LoG}}(u, v) = -4\pi^2 D^2 e^{-2\pi^2\sigma^2 D^2}. \quad (6)$$

This is a band-pass filter. It is zero at $D = 0$, rises to a peak, then decays, so it keeps edge scale frequencies and rejects both the flat background and high frequency noise. Edges are the zero crossings of $\mathcal{F}^{-1}\{F H_{\text{LoG}}\}$, the Marr and Hildreth criterion. A step edge becomes a positive then negative swing in the second derivative, and the crossing between them marks the edge. The parameter σ sets the scale of the edges detected.

5.3 Region segmentation through the frequency domain

A uniform region is dominated by low frequencies. Multiplying by a Gaussian low-pass filter

$$H_{\text{lp}}(u, v) = e^{-D^2/(2D_0^2)}, \quad f_{\text{lp}} = \mathcal{F}^{-1}\{F H_{\text{lp}}\}, \quad (7)$$

removes texture and noise so each region becomes nearly flat, and then a global threshold such as Otsu separates the regions. This is the same smoothing then threshold idea used for the thermal image in Q2, now written as a spectral multiplication. Since $H_{\text{hp}} = 1 - H_{\text{lp}}$, one transform gives both: high-pass pulls out the boundaries between regions and low-pass pulls out the smooth interior of regions, and a closed high-pass contour encloses a low-pass region.

Texture regions with a Gabor filter bank. When regions differ by texture rather than brightness, a single threshold fails, but different textures sit in different parts of the spectrum. A Gabor filter is a Gaussian multiplied by a complex exponential, $h(x, y) = g_\sigma(x, y) e^{j2\pi(u_0 x + v_0 y)}$, with the transform

$$H(u, v) = e^{-2\pi^2\sigma^2[(u-u_0)^2 + (v-v_0)^2]}, \quad (8)$$

a Gaussian window centred on frequency (u_0, v_0) . Each Gabor is a band-pass filter that selects one orientation and scale. Filtering the image with a bank of Gabors at several (u_0, v_0) gives every pixel a vector of responses, a local frequency signature. Pixels with similar signatures are grouped, by thresholding or clustering, into texture regions. This is the frequency domain route to segmenting regions that intensity alone cannot separate.

6 How to run

```
pip install -r requirements.txt
streamlit run app.py                # web app (Q1, Q2, comparison, Q3 theory)
```

or from the command line:

```
python -m src.rgb_segment      --image data/person_rgb.png
python -m src.thermal_segment  --image data/person_thermal.png
python -m src.compare
python -m src.fourier          --image data/person_rgb.png
```

Data. RGB: the public domain astronaut image (NASA astronaut Eileen Collins) from scikit-image. Thermal: a frame from the OSU Thermal Pedestrian Database (OTCBVS Benchmark).

Note on SAM. SAM2’s checkpoints and demo were not reachable from the machine used to build this, so the reference is the SAM3 demo, the current model in the same Segment Anything family, run with the prompt **person**. The comparison does not depend on the exact version and can be redone with the official SAM2 demo (see the README).

References

- [1] C. Rother, V. Kolmogorov, A. Blake, “GrabCut: interactive foreground extraction using iterated graph cuts,” *ACM SIGGRAPH*, 2004.
- [2] D. Marr, E. Hildreth, “Theory of edge detection,” *Proc. Royal Society of London B*, 1980.
- [3] R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, Pearson (frequency domain filtering).
- [4] A. Kirillov et al., “Segment Anything,” *ICCV*, 2023; N. Ravi et al., “SAM 2: Segment Anything in Images and Videos,” 2024.
- [5] J. W. Davis, M. A. Keck, “A two-stage template approach to person detection in thermal imagery” (OSU Thermal Pedestrian Database, OTCBVS Benchmark), *WACV*, 2005.

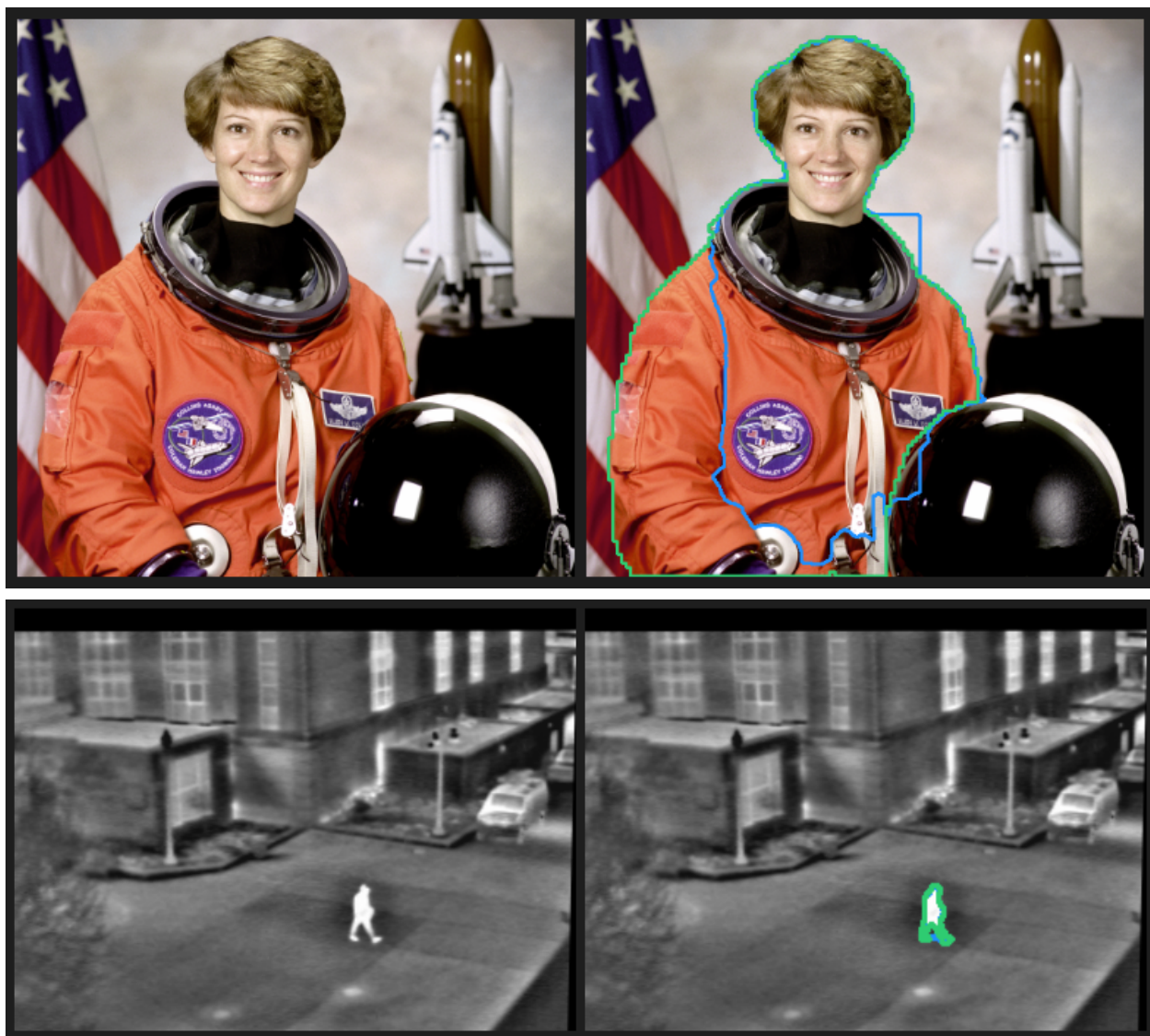


Figure 3: Blue is our classical boundary, green is SAM. Top row is the RGB image, bottom row is the thermal image.

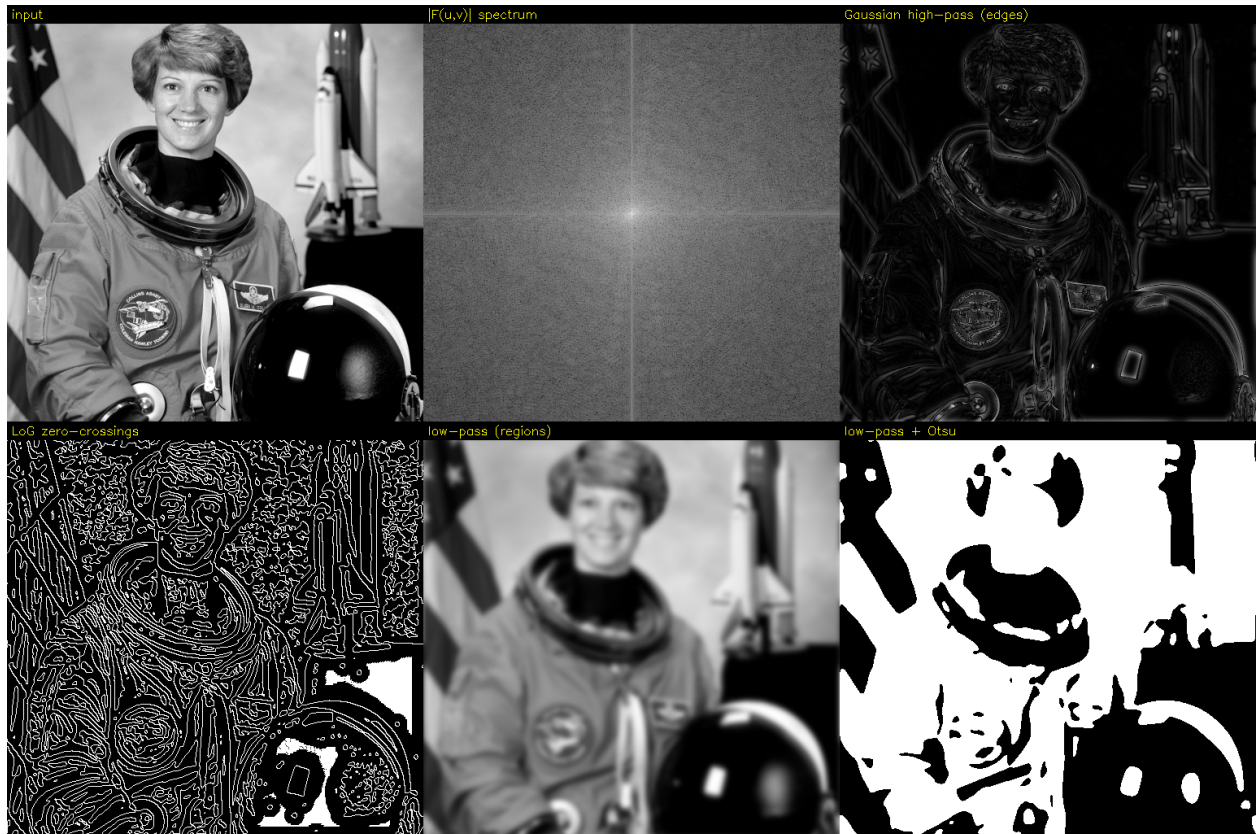


Figure 4: Frequency domain demonstration (`src/fourier.py`). Top row: input, log magnitude spectrum $|F(u, v)|$, and a Gaussian high-pass result (edges). Bottom row: LoG zero crossings, a Gaussian low-pass result (smoothed regions), and low-pass followed by an Otsu threshold (region segmentation).